

dj



Persistence

Les sessions DJANGO

- **Présentation**

- Django prend complètement en charge les sessions anonymes.
- Les données sont stockées côté serveur
- l'envoi et la réception des cookies sont transparents.
- Les cookies contiennent un identifiant de session, avec le modèle `django.contrib.sessions.models.Session`
- Les sessions sont gérées par Django par défaut au moyen d'un `middleware` `django.contrib.sessions.middleware.SessionMiddleware` de l'application `django.contrib.sessions`
- Par défaut, Django stocke les sessions dans la base de données.
- On peut modifier ce comportement pour placer les sessions dans le système de fichier :
`SESSION_ENGINE = "django.contrib.sessions.backends.cache"`
- ou dans un cache (bonne pratique) : `SESSION_ENGINE = "django.contrib.sessions.backends.file"`

Les sessions DJANGO

- Utilisation dans les vues
- l'objet `session` est disponible comme attribut de l'objet request dans toutes les vues
- En paramètre dans les vus fonctions
- Dans `self.request` dans les vues basée sur une classe
- session est doté d'un comportement de dictionnaire, il est conseillé de l'utiliser comme tel

```
__getitem__(): request.session['fav_color']  
__setitem__(): request.session['fav_color'] = 'blue'  
__contains__(): 'fav_color' in request.session  
__delitem__(): del request.session['fav_color']
```

```
keys(), items(), get(), pop(), clear()...
```

Les sessions DJANGO

- Sauvegarder la session
- La session sauvegarde ses données [en base / dans un fichier/ dans un cookie / en cache] quand la valeur d'une ses clés est modifiée
- Si une clé a pour valeur une structure imbriquée, il faut executer `session.modified = True` pour activer la sauvegarde

```
request.session['my_car'] = 'mini' # sauvegarde
```

```
request.session['my_car']['color'] = 'red' # pas de sauvegarde si color existait déjà  
request.session.modified = True # sauvegarde
```

Les sessions DJANGO

- Méthodes de gestion de session
 - `flush()` : Supprime les données de la session actuelle et supprime le cookie de session. En liaison avec `django.contrib.auth.logout()`.
 - `set_test_cookie()` : Crée un cookie de test pour déterminer si le navigateur de l'utilisateur prend en charge les cookies.
 - `test_cookie_worked()` : Renvoie True ou False, selon que le navigateur de l'utilisateur a accepté le cookie de test ou non
 - `delete_test_cookie()` : Supprime le cookie de test. Utilisé pour faire le ménage derrière soi.
 - `get_session_cookie_age()` : Renvoie l'âge des cookies de session, en secondes. La valeur par défaut est `SESSION_COOKIE_AGE`.
 - `set_expiry(value)` : Définit la durée d'expiration de la session. Il est possible de passer plusieurs valeurs différentes, entiers ou objets de date

Les sessions DJANGO

- Méthodes de gestion de la persistance
 - `get_expire_at_browser_close()` : Renvoie True ou False selon que le cookie de session utilisateur expire au moment de la fermeture du navigateur Web de l'utilisateur ou non. Lié à la variable de configuration `SESSION_EXPIRE_AT_BROWSER_CLOSE`
 - `clear_expired()` : Supprime les sessions expirées du stockage des sessions.
 - `cycle_key()` : Crée une nouvelle clé de session tout en conservant les données de session actuelles. `django.contrib.auth.login()` appelle cette méthode pour lutter contre les attaques par fixation de session.

Les sessions DJANGO

- Utilisation en dehors des vues
- On peut utiliser la classe `django.contrib.sessions.backends.db.SessionStore`

```
from django.contrib.sessions.backends.db import SessionStore

# création
s = SessionStore()
s['last_login'] = 1376587691
s.create()
print(s.session_key) # '2b1189a188b44ad18c35e113ac6ceead'

# chargement
s = SessionStore(session_key='2b1189a188b44ad18c35e113ac6ceead')
print(s['last_login']) # 1376587691
```

Les caches DJANGsO

- Principe
- Mettre en cache les pages d'une application web revient à implémenter l'algorithme suivant

Soit une URL

si la page correspondante est dans le cache :

retourne la page du cache

sinon:

génère la page

enregistre la page générée dans le cache pour une période donnée

retourne la page générée

Les caches DJANGsO

- Configuration
- Django met à disposition la variable de configuration CACHES qui s'écrit comme ci dessous

```
CACHES = {  
    'default': {  
        'BACKEND': 'chemin.python.vers.classe',  
        'LOCATION': 'ip:port ou chemin/vers/socket.unix',  
    }  
}
```

Les BACKEND possibles par défaut sont

- Memcached : cache en mémoire, rapide, mais non persistant
- Base de donnée : persistant, mais lent. Bases de données dédiées au cache en lecture/écriture
- Système de fichier, via une serialisation

Les caches DJANGsO

- Memcached
- Avec Memcached, LOCATION peut accepter une liste de serveurs au lieu d'un seul pour répartir le cache

```
CACHES = {  
    'default': {  
        'BACKEND': 'django.core.cache.backends.memcached.MemcachedCache',  
        'LOCATION': '127.0.0.1:11211',  
    }  
}
```

Les caches DJANGsO

- Base de Donnée
- Avec un cache en Base de Données, LOCATION prend comme valeur le nom d'une table spécifique au cache à créer par Django

```
CACHES = {  
    'default': {  
        'BACKEND': 'django.core.cache.backends.db.DatabaseCache',  
        'LOCATION': 'my_cache_table',  
    }  
}
```

```
# dans un terminal  
py manage.py createcachetable
```

Les caches DJANGsO

- Pseudo Cache de Dev
- Le moteur Dummy permet de tester les configuration de mises en cache sans pour autant activer un cache préjudiciable en développement

```
CACHES = {  
    'default': {  
        'BACKEND': 'django.core.cache.backends.dummy.DummyCache',  
    }  
}
```

Les caches DJANGsO

- Paramètres de cache

Des variables de configurations sont disponibles pour contrôler le comportement du cache:

- TIMEOUT: le délai d'expiration du cache par défaut, en secondes.
 - 300 par défaut.
 - None implique que le cache n'expire jamais.
 - 0 implique que les clés expirent immédiatement (pas de cache).
- OPTIONS: options spécifiques à chaque backend
- MAX_ENTRIES: nombre max d'entrées autorisées avant suppression des valeurs plus anciennes.
 - 300 par défaut.
- CULL_FREQUENCY: proportion d'éléments effacés lorsque MAX_ENTRIES est atteint ($1/\text{CULL_FREQUENCY}$).
 - 3 par défaut
 - 0 purge tout le cache
- KEY_PREFIX: préfixe par défaut des clés de cache.
- VERSION: le numéro de version par défaut des clés de cache.
- KEY_FUNCTION: chaîne contenant un chemin pointé vers une fonction qui définit la manière de composer un préfixe, une version et une clé pour former une clé de cache finale.

Les caches DJANGsO

- **Stratégies de cache : cache par site**
- Stratégie visant à mettre en cache tout le site.
- Réglages (dans cet ordre précis) :

```
MIDDLEWARE = [  
    'django.middleware.cache.UpdateCacheMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.cache.FetchFromCacheMiddleware',  
    ...  
]
```

L'alias de cache à utiliser pour le stockage.

CACHE_MIDDLEWARE_ALIAS = ...

le nombre de secondes durant lequel chaque page doit être conservée en cache.

CACHE_MIDDLEWARE_SECONDS = ...

prefixes des pages du site si cache multi site

CACHE_MIDDLEWARE_KEY_PREFIX = ...

Les caches DJANGsO

- **Stratégies de cache : cache par site**
- Algorithme des Middlewares :
- **FetchFromCacheMiddleware** met en mémoire cache les réponses GET et HEAD ayant le statut 200, pour autant que les en-têtes de requête et de réponse le permettent.
- Les réponses aux requêtes à une même URL ayant des paramètres de requête différents sont considérées comme des pages distinctes et sont mises en cache séparément.
- **FetchFromCacheMiddleware** s'attend à ce que la réponse à une requête HEAD possède les mêmes en-têtes de réponse que la requête GET équivalente ; et dans ce cas, il peut renvoyer une réponse GET à partir du cache lors d'une requête HEAD.
- **UpdateCacheMiddleware** définit automatiquement quelques en-têtes dans chaque réponse HttpResponse, lesquels affectent les caches amonts:
- L'en-tête **Expires** est défini à la date/heure courante additionnée de **CACHE_MIDDLEWARE_SECONDS**.
- L'en-tête **Cache-Control** est défini à un âge maximal de la page, aussi selon le réglage **CACHE_MIDDLEWARE_SECONDS**

Les caches DJANGsO

- **Stratégies de cache : cache par vue**
- Une manière plus fine d'utiliser l'infrastructure de cache est de mettre en cache le résultat de vues individuelles.
- `django.views.decorators.cache` définit un décorateur `cache_page` qui se charge de mettre en cache la réponse de la vue

```
# dans la vue...
from django.views.decorators.cache import cache_page

@cache_page(60 * 15, key_prefix='my_app')
def my_view(request):
    ...

# ... ou dans la conf url
from django.views.decorators.cache import cache_page

urlpatterns = [
    path('foo/<int:code>/', cache_page(60 * 15)(my_view)),
]
```

Les caches DJANGsO

- **Stratégies de cache : cache par template**
- il est encore possible de mettre en cache des portions de templates avec la balise de gabarit `{% cache %}`.
- Pour que les templates aient accès à cette balise, il faut placer `{% load cache %}` au début du template.

```
% load cache %  
{% cache 500 sidebar %}  
.. sidebar ..  
{% endcache %}
```

Les caches DJANGsO

- **Stratégies de cache : cache bas niveau**
- Le module `django.core.cache.cache` fournit une Api permettant aux developpeurs de mettre dans le cache configuré dans CACHES des objets de django (querySets, listes, dictionnaires, objets...).

```
from django.core.cache import cache
```

```
# met value dans key pendant 30s  
cache.set('key', 'value', 30)
```

```
# renvoie la valeur de key ou valeur par défaut si expiré  
cache.get('key', 'default_value_if_expired')
```

```
#met à jour la date d'expiration à 100s  
cache.touch('key', 100)
```

```
#suppression de la clé  
cache.delete('key')
```

```
# suppression de TOUT LE CACHE  
cache.clear()
```